Procedimiento de Implantación del Software

Plataforma EcoRuteando — Sistema de Movilidad Sostenible

Control de versiones del documento

| Versión | Fecha | Autor | Descripción del cambio | |---------|-------|-------|----------------------| | 1.0 | 13 de octubre de 2026 | Equipo DevOps y Arquitectura | Emisión inicial del procedimiento de implantación |

Metadatos del documento

| Campo | Valor | |-------|-------| | **Nombre del documento** | Procedimiento de Implantación del Software | | **Proyecto** | EcoRuteando — Plataforma de Movilidad Sostenible | | **Versión del software** | 1.0 | | **Clasificación** | Documentación oficial — Confidencial interno | | **Marco de referencia** | Competencia: Implantación del Software | | **Estado** | Vigente | | **Próxima revisión** | Trimestral |

Metadatos del proyecto

| Campo | Valor | |-------|-------| | **Nombre del sistema** | EcoRuteando | | **Descripción** | Plataforma web y móvil para planificación de rutas ecológicas, estimación de impacto ambiental (CO_2, calorías), reporte de obstáculos urbanos y alertas climáticas | | **Backend** | API REST .NET 9 (C#) — 23 proyectos, arquitectura DDD + Clean Architecture con CQRS/MediatR | | **Frontend Web** | Aplicación SPA React 18 + Vite 8 + Tailwind CSS | | **Aplicación Móvil** | Expo SDK 57 / React Native 0.86 (Android, iOS, Web) | | **Base de Datos** | PostgreSQL 16 + PostGIS 3.5 + Liquibase 5.0.2 | | **Bounded Contexts** | Security, Mobility, Notifications, Admin, Integrations | | **Esquemas de BD** | security, mobility, community, admin, integrations, notifications (6 esquemas) | | **Métodos de autenticación** | Email/contraseña + OAuth Google | | **Entornos** | Desarrollo local (Docker), QA, Staging, Producción | | **Plataformas cloud** | Render (backend + BD), Vercel (frontend), EAS/Expo (móvil) | | **Estado funcional** | 87% de funcionalidades completadas |

---

1. Objetivo y alcance de la implantación

1.1 Objetivo

Establecer el procedimiento formal, repetible y auditado para la **implantación del software EcoRuteando** en el entorno de producción, garantizando:

1. La **disponibilidad continua** del sistema sin pérdida de datos
2. La **integridad** del esquema de base de datos y los datos de usuarios
3. La **seguridad** de las credenciales y secretos en todos los entornos
4. La **trazabilidad** completa de cada operación de despliegue
5. La **reversibilidad** del proceso ante cualquier fallo crítico
6. El **cumplimiento** de los criterios de aceptación definidos por el cliente

1.2 Alcance

El presente procedimiento cubre la implantación de los **cuatro componentes** del sistema:

| # | Componente | Tecnología | Plataforma de despliegue | Entorno de producción | |---|---|---|---|---| | 1 | Backend API | .NET 9 (Docker) | Render | `https://ecoruteando-backend.onrender.com` | | 2 | Base de Datos | PostgreSQL 16 + PostGIS + Liquibase | Render PostgreSQL | Gestiónada (managed) | | 3 | Frontend Web | React + Vite + nginx | Vercel | Dominio del proyecto | | 4 | Aplicación Móvil | Expo / React Native | EAS Build (APK/IPA) | Distribución interna / tiendas |

1.3 Fuera de alcance

- Configuración de dominios DNS y certificados SSL (responsabilidad del cliente)
- Contratación de servicios cloud (planes de pago)
- Publicación en Google Play Store / Apple App Store (configuración `submit` vacía en `eas.json`)
- Migración de datos de sistemas externos previos (el sistema es de nueva implantación)

1.4 Definiciones y abreviaturas

| Término | Definición | |---|---| | **UAT** | User Acceptance Testing — Pruebas de aceptación de usuario | | **RTO** | Recovery Time Objective — Tiempo máximo aceptable de inactividad | | **RPO** | Recovery Point Objective — Pérdida máxima aceptable de datos | | **Rollback** | Reversión del despliegue a la versión anterior estable | | **Changeset** | Unidad atómica de migración en Liquibase | | **Healthcheck** | Verificación periódica de disponibilidad de un servicio | | **Ventana de mantenimiento** | Período programado de inactividad para operaciones de despliegue |

---

2. Responsables y roles

2.1 Matriz de responsabilidades (RACI)

| Actividad | DevOps Engineer | Desarrollador Líder | QA Engineer | Cliente / Gestor | |---|---|---|---|---| | Aprovisionamiento de infraestructura | **R** | C | I | A | | Configuración de secretos y variables de entorno | **R** | C | I | I | | Backup preventivo de base de datos | **R** | C | I | A | | Ejecución de migraciones Liquibase | C | **R** | I | A | | Despliegue de Backend (Docker/Render) | **R** | C | I | I | | Despliegue de Frontend (Vercel) | **R** | C | I | I | | Build de App Móvil (EAS) | C | **R** | I | I | | Pruebas de humo y salud | I | C | **R** | I | | Pruebas de aceptación (UAT) | I | C | **R** | **A** | | Acta de entrega formal | C | C | C | **R/A** | | Capacitación a usuarios | C | **R** | I | **A** | | Monitoreo post-implantación | **R** | C | I | I | | Resolución de incidentes críticos | **R** | **R** | C | I |

Leyenda: R = Responsible (ejecuta) | A = Accountable (aprueba) | C = Consulted (consulta) | I = Informed (informado)

2.2 Descripción de roles

DevOps Engineer

- Responsable de la infraestructura cloud (Render, Vercel, EAS)
- Aprovisionamiento y gestión de secretos y variables de entorno
- Ejecución de backups y restauraciones
- Despliegue del software y monitoreo de disponibilidad
- Gestión de pipelines CI/CD

Desarrollador Líder (Tech Lead)

- Responsable técnico de la calidad del código desplegado
- Ejecución y supervisión de migraciones de base de datos (Liquibase)
- Resolución de incidencias técnicas durante el despliegue
- Capacitación técnica al equipo de soporte

QA Engineer

- Diseño y ejecución de pruebas de humo, integración y aceptación
- Verificación de criterios de aceptación post-despliegue
- Generación de reportes de defectos
- Aprobación técnica de la calidad del despliegue

Cliente / Gestor del Proyecto

- Aprobación final del acta de entrega
- Validación de funcionalidades en el entorno de producción (UAT)
- Autorización de ventanas de mantenimiento
- Recepción formal del software implantado

---

3. Fase 1: Planeación de la implantación

3.1 Verificación de hardware y software del servidor

Antes de proceder con cualquier despliegue, se verifica que la infraestructura cloud cumple con los requisitos mínimos del sistema.

3.1.1 Requisitos de infraestructura — Backend (Render)

| Componente | Requisito mínimo | Plan recomendado | Verificación | |----------|----------------|----------|----------| | **Runtime** | .NET 9 (Docker) | Starter o superior | `docker --version` en build | | **CPU** | 0.5 vCPU | 1 vCPU | Dashboard Render | | **RAM** | 512 MB | 1 GB | Dashboard Render | | **Disco** | 1 GB | 2 GB | Dashboard Render | | **Puerto** | 5124 (`ASPNETCORE_URLS`) | — | Variable de entorno | | **Conexión saliente** | Habilitada (Google Maps, SMTP) | — | Prueba de conectividad |

3.1.2 Requisitos de infraestructura — Base de Datos (Render PostgreSQL)

| Componente | Requisito mínimo | Plan recomendado | Verificación | |----------|----------------|----------|----------| | **Motor** | PostgreSQL 16 | PostgreSQL 16 | Dashboard Render | | **Extensión**

PostGIS | PostGIS 3.5 | PostGIS 3.5 | `\dx` en psql | | **Almacenamiento** | 1 GB | 5 GB | Dashboard Render | | **Conexiones máx.** | 20 | 50 | Dashboard Render | | **Backups** | Automáticos diarios | Habilitados | Dashboard Render | | **Acceso externo** | Habilitado (para Liquibase) | IP whitelist | Dashboard Render |

3.1.3 Requisitos de infraestructura — Frontend (Vercel)

| Componente | Requisito mínimo | Plan recomendado | Verificación | |----------|----------------|----------|----------| | **Build** | Node.js 22 + Vite | — | `node -v` | | **Almacenamiento estático** | 100 MB | 500 MB | Dashboard Vercel | | **Ancho de banda** | 100 GB/mes | Ilimitado | Dashboard Vercel | | **HTTPS** | Automático (Let's Encrypt) | — | Verificar en navegador | | **Rewrites** | `/api/*` → Backend | — | `vercel.json` |

3.1.4 Requisitos de infraestructura — Móvil (EAS)

| Componente | Requisito mínimo | Plan recomendado | Verificación | |----------|----------------|----------|----------| | **Node.js** | 22.x | 22.x LTS | `node -v` | | **Expo SDK** | 57.x | 57.x | `npx expo --version` | | **EAS CLI** | Latest | Latest | `eas --version` | | **Cuenta Expo** | Gratuita o de pago | De pago (builds concurrentes) | `eas whoami` |

3.1.5 Checklist de verificación pre-despliegue

```
 # 1. Verificar acceso a Render
curl -s https://ecoruteando-backend.onrender.com/api/test/public

# 2. Verificar acceso a Vercel
curl -sI https://<dominio-vercel>.vercel.app

# 3. Verificar conectividad a PostgreSQL desde Render
# (se realiza desde el dashboard de Render → Logs)

# 4. Verificar cuenta EAS
eas whoami

# 5. Verificar Google Cloud Console (API keys activas)
# (revisión manual en console.cloud.google.com)

# 6. Verificar Gmail SMTP (credenciales válidas)
# (prueba de envío desde el dashboard de Render)
```

3.2 Cronograma de ventanas de mantenimiento

3.2.1 Definición de ventanas

| Tipo de mantenimiento | Duración | Frecuencia | Horario sugerido | |----------|----------|----------|----------| | **Mantenimiento menor** (configuración, patches) | 15-30 min | Semanal | Martes 02:00 - 03:00 (GMT-5) | | **Mantenimiento mayor** (migraciones, features) | 1-2 horas | Quincenal | Sábado 01:00 - 04:00 (GMT-5) | | **Mantenimiento de emergencia** | Variable | Bajo demanda | Cualquier horario con autorización | | **Ventana de despliegue inicial** | 4-6 horas | Única | Sábado 01:00 - 07:00 (GMT-5) |

3.2.2 Cronograma del despliegue inicial

| # | Actividad | Duración acumulada | Hora inicio | Responsable | |---|---|---|---|---| | T-24h | Backup preventivo completo de BD | 24 h antes | — | DevOps | | T-2h | Notificación a usuarios del mantenimiento | 2 h antes | 23:00 | Cliente | | T-0 | Inicio de ventana de mantenimiento | 0:00 | 01:00 | DevOps | | T+0:15 | Backup preventivo final | +15 min | 01:15 | DevOps | | T+0:30 | Ejecución de migraciones Liquibase | +30 min | 01:30 | Tech Lead | | T+1:00 | Despliegue de Backend | +1 h | 02:00 | DevOps | | T+1:30 | Despliegue de Frontend | +1.5 h | 02:30 | DevOps | | T+2:00 | Pruebas de humo | +2 h | 03:00 | QA | | T+2:30 | Pruebas de integración | +2.5 h | 03:30 | QA | | T+3:00 | Pruebas de aceptación (UAT) | +3 h | 04:00 | QA + Cliente | | T+4:00 | Build y distribución de App Móvil | +4 h | 05:00 | Tech Lead | | T+5:00 | Cierre de ventana / Go-Live | +5 h | 06:00 | DevOps + Cliente | | T+24h | Verificación post-implantación | +24 h | — | DevOps + QA |

3.2.3 Comunicación a usuarios

Notificación previa (24 horas antes):

> Estimados usuarios: El sistema EcoRuteando estará fuera de servicio el [FECHA] de [HORA INICIO] a [HORA FIN] (GMT-5) por mantenimiento programado. Durante este período no podrá acceder a la plataforma. Disculpe las molestias.

Notificación de finalización:

> Estimados usuarios: El mantenimiento de EcoRuteando ha sido completado exitosamente. El sistema está disponible nuevamente en [URL]. Agradecemos su paciencia.

---

4. Fase 2: Preparación del entorno y plataforma tecnológica

4.1 Aprovisionamiento de servicios cloud

4.1.1 Render — Backend API

| Paso | Acción | Detalle | |---|---|---| | 1 | Crear cuenta en Render | render.com | | 2 | Crear Web Service | Nuevo → "Web Service" → Conectar repositorio GitHub | | 3 | Configurar build | Build Command: `dotnet publish src/Api/EcoRuteando.Api -c Release -o ./output` | | 4 | Configurar start | Start Command: `./output/EcoRuteando.Api` | | 5 | Configurar puerto | Puerto: `5124` | | 6 | Configurar Docker | Dockerfile Path: `dockerfile` (si se usa Docker) | | 7 | Configurar health check | Health Check Path: `/api/test/public` | | 8 | Asignar variables de entorno | Ver sección 4.2 | | 9 | Activar auto-deploy | Deploy on push: `main` branch |

4.1.2 Render — PostgreSQL

| Paso | Acción | Detalle | |---|---|---| | 1 | Crear Base de Datos | Dashboard → New → PostgreSQL | | 2 | Configurar nombre | Database Name: `ecoRuteando_db` | | 3 | Configurar versión | PostgreSQL Version: `16` | | 4 | Habilitar PostGIS | Extension: `postgis` (requiere plan de pago) | | 5 | Habilitar backups | Backups: Diarios automáticos | | 6 | Copiar connection string | Dashboard →

Connection → External Database URL | | 7 | Configurar IP whitelist | Agregar IP del servicio backend o `0.0.0.0/0` (menos seguro) |

4.1.3 Vercel — Frontend Web

| Paso | Acción | Detalle | |------|--------|---------| | 1 | Crear cuenta en Vercel | vercel.com | | 2 | Importar proyecto | New Project → Conectar repositorio GitHub | | 3 | Configurar framework | Framework Preset: `Vite` | | 4 | Configurar build | Build Command: `npm run build` | | 5 | Configurar output | Output Directory: `dist` | | 6 | Configurar rewrites | Ver `vercel.json` (sección 4.1.4) | | 7 | Asignar variables de entorno | Ver sección 4.2 | | 8 | Configurar dominio | Settings → Domains → Agregar dominio personalizado | | 9 | Activar auto-deploy | Production Branch: `main` |

4.1.4 Configuración `vercel.json`

```
{
  "rewrites": [
    {
      "source": "/api/:match*",
      "destination": "https://ecoruteando-backend.onrender.com/api/:match*"
    },
    {
      "source": "/((?!api/).*)",
      "destination": "/index.html"
    }
  ]
}
```

4.1.5 EAS — Aplicación Móvil

| Paso | Acción | Detalle | |------|--------|---------| | 1 | Crear cuenta Expo | expo.dev | | 2 | Configurar proyecto | `eas init` en directorio del proyecto | | 3 | Configurar eas.json | Ver sección 4.2.3 | | 4 | Autenticar CLI | `eas login` | | 5 | Configurar builds | `eas build:configure` |

4.2 Aprovisionamiento de variables de entorno y secretos

4.2.1 Variables de entorno — Backend (Render)

| Variable | Descripción | Fuente | Obligatoria | |----------|-------------|--------|-------------| | `ConnectionStrings__DefaultConnection` | Connection string de PostgreSQL | Render PostgreSQL | ✅ | | `Jwt__SecretKey` | Clave secreta JWT (≥32 bytes) | Generada: `openssl rand -base64 64` | ✅ | | `Jwt__Issuer` | Emisor del token | Literal: `EcoRuteando` | ✅ | | `Jwt__Audience` | Audiencia del token | Literal: `EcoRuteandoUsers` | ✅ | | `Jwt__ExpirationMinutes` | Expiración del token | Literal: `60` | ✅ | | `GoogleMaps__ApiKey` | API key de Google Maps | Google Cloud Console | ✅ | | `Google__ClientId` | OAuth Client ID Google | Google Cloud Console | ✅ | | `Google__ClientSecret` | OAuth Client Secret Google | Google Cloud Console | ✅ | | `EmailSettings__Host` | Servidor SMTP | Literal: `smtp.gmail.com` | ✅ | | `EmailSettings__Port` | Puerto SMTP | Literal: `587` | ✅ | | `EmailSettings__EnableSsl` | Habilitar SSL | Literal: `true` | ✅ | | `EmailSettings__Username` | Usuario SMTP | Gmail | ✅ | | `EmailSettings__Password` | Contraseña de aplicación | Gmail (16 caracteres) | ✅ | | `EmailSettings__FromName` | Nombre del remitente | Literal: `EcoRuteando` | ✅ | |

`EmailSettings__FromEmail` | Email del remitente | Gmail | ✅ | | `Admin__Email` | Email del admin seed | Literal | ✅ | | `Admin__Password` | Contraseña del admin seed | Generada segura | ✅ | | `ASPNETCORE_ENVIRONMENT` | Entorno | Literal: `Production` | ✅ | | `ASPNETCORE_URLS` | URL de escucha | Literal: `http://+:5124` | ✅ |

4.2.2 Variables de entorno — Frontend (Vercel)

| Variable | Descripción | Fuente | Obligatoria | |----------|-------------|--------|-------------| | `VITE_GOOGLE_MAPS_API_KEY` | API key JavaScript Maps | Google Cloud Console | ✅ | | `VITE_GOOGLE_CLIENT_ID` | OAuth Client ID Google | Google Cloud Console | ⚠️ Opcional |

4.2.3 Variables de entorno — Móvil (eas.json)

```
{
  "build": {
    "development": {
      "env": {
        "EXPO_PUBLIC_API_URL": "http://localhost:5124"
      },
      "developmentClient": true
    },
    "preview": {
      "env": {
        "EXPO_PUBLIC_API_URL": "https://ecoruteando-backend.onrender.com"
      },
      "distribution": "internal"
    },
    "production": {
      "env": {
        "EXPO_PUBLIC_API_URL": "https://ecoruteando-backend.onrender.com"
      }
    }
  },
  "submit": {
    "production": {}
  }
}
```

4.2.4 Gestión segura de secretos

| Regla | Implementación | |-------|---------------| | **Nunca** commitear secretos | `.env` en `.gitignore` | | **Nunca** poner secretos en `docker-compose.yml` | Usar `env_file` o variables del host | | Rotación de API keys | Cada 90 días o ante sospecha de compromiso | | Acceso mínimo | Cada servicio solo recibe las variables que necesita | | Auditoría | Secret Scanning en GitHub habilitado |

4.2.5 Configuración de Google Cloud Console

| Paso | Acción | |------|--------| | 1 | Crear proyecto en console.cloud.google.com | | 2 | Habilitar APIs: Maps JavaScript API, Directions API, Geocoding API, Places API | | 3 | Crear API Key (para backend y frontend) | | 4 | Restringir API Key por HTTP referrer y/o IP | | 5 | Crear OAuth Client ID (Web

application) | | 6 | Configurar Authorized redirect URIs con el dominio de producción | | 7 | Copiar Client ID y Client Secret al panel de Render/Vercel |

---

5. Fase 3: Migración y respaldo de datos

5.1 Plan de backup preventivo

5.1.1 Objetivos del backup

| Objetivo | Valor | Justificación | |----------|-------|---------------| | **RPO** (pérdida máxima aceptable) | 0 datos | Backup completo antes de cada despliegue | | **RTO** (recuperación máxima aceptable) | 30 minutos | Restauración desde backup reciente | | **Retención** | 30 días | Cumplimiento de políticas de auditoría | | **Almacenamiento** | Mínimo 2 ubicaciones | Local (Render) + externo (S3/GCS o descarga local) |

5.1.2 Procedimiento de backup preventivo (pre-despliegue)

Paso 1: Backup completo con pg_dump

```
 #!/bin/bash
# scripts/backup-predeploy.sh
# Ejecutar ANTES de cada despliegue a producción

set -e

# Configuración
DB_HOST="localhost"        # O host de Render
DB_PORT="5432"
DB_NAME="ecoRuteando_db"
DB_USER="postgres"
BACKUP_DIR="./backups"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/ecoruteando_predeploy_${TIMESTAMP}.sql.gz"

# Crear directorio de backups
mkdir -p ${BACKUP_DIR}

echo "=== BACKUP PRE-DESPLIEGUE ==="
echo "Fecha: $(date)"
echo "Base de datos: ${DB_NAME}"
echo "Archivo: ${BACKUP_FILE}"
echo ""

# Ejecutar backup completo
pg_dump \
  -h ${DB_HOST} \
  -p ${DB_PORT} \
  -U ${DB_USER} \
  -d ${DB_NAME} \
  --format=custom \
  --compress=9 \
  --verbose \
```

```
  --file=${BACKUP_FILE}

# Verificar integridad del backup
FILESIZE=$(stat -f%z "${BACKUP_FILE}" 2>/dev/null || stat -c%s "${BACKUP_FILE}")
MIN_SIZE=1048576  # 1 MB mínimo

if [ "${FILESIZE}" -lt "${MIN_SIZE}" ]; then
    echo "ERROR: Backup sospechosamente pequeño (${FILESIZE} bytes)"
    exit 1
fi

# Verificar que el backup puede listar contenido
pg_restore --list ${BACKUP_FILE} > /dev/null 2>&1
if [ $? -ne 0 ]; then
    echo "ERROR: Backup corrupto — no se puede listar contenido"
    exit 1
fi

echo ""
echo "=== BACKUP COMPLETADO EXITOSAMENTE ==="
echo "Tamaño: $(du -h ${BACKUP_FILE} | cut -f1)"
echo "SHA256: $(sha256sum ${BACKUP_FILE} | cut -d' ' -f1)"
echo "Archivo: ${BACKUP_FILE}"
```

Paso 2: Backup de esquema (DDL)

```
 # Backup solo del esquema (estructura)
pg_dump \
  -h ${DB_HOST} -p ${DB_PORT} -U ${DB_USER} -d ${DB_NAME} \
  --schema-only \
  --file=${BACKUP_DIR}/schema_${TIMESTAMP}.sql
```

Paso 3: Verificación de integridad

```
 # Restaurar backup en BD temporal para verificar
createdb ecoruteando_verify_test
pg_restore -d ecoruteando_verify_test ${BACKUP_FILE}

# Contar tablas
psql -d ecoruteando_verify_test -c "SELECT COUNT(*) FROM information_schema.tables WHERE ta

# Verificar Liquibase
psql -d ecoruteando_verify_test -c "SELECT COUNT(*) FROM databasechangelog;"

# Limpiar
dropdb ecoruteando_verify_test
```

5.1.3 Procedimiento de backup automático programado

Se configura un cron job para backups automáticos:

```
 # crontab -e
# Backup diario a las 02:00
```

```
0 2 * * * /path/to/scripts/backup-database.sh >> /var/log/ecoruteando-backups.log 2>&1

# Backup semanal completo (domingo 03:00)
0 3 * * 0 /path/to/scripts/backup-full-weekly.sh >> /var/log/ecoruteando-backups.log 2>&1
```

5.2 Ejecución de migraciones con Liquibase

5.2.1 Estructura del changelog maestro

El esquema de la base de datos está definido en **77 changesets** organizados en:

```
EcoRuteandoDb/
├── changelog/
│   └── db.changelog-master.yaml    # Master changelog (incluye todos los módulos)
├── 01_ddl/
│   ├── 01_extensions/              # postgis, pgcrypto, unaccent, pg_trgm, citext
│   ├── 02_schemas/                 # security, mobility, admin, community, integrations, n
│   ├── 03_tables/                  # 36 tablas en 6 esquemas
│   └── 04_indexes_constraints/     # Índices, foreign keys, unique constraints
└── 02_dml/
    └── seeds/                      # Roles, permisos, política de seguridad, factores de t
```

5.2.2 Migración de los 6 esquemas

| # | Esquema | Tablas | Changesets clave | |---|---------|--------|-----------------| | 1 | `security` | 14 | Users, roles, permissions, refresh_tokens, sessions, 2FA, audit_logs, security_policies | | 2 | `mobility` | 10 | Routes, POIs, favorites, ratings, shared_routes, trip_history, transport_factors | | 3 | `community` | 5 | Obstacle_reports, report_votes, alerts, alert_route, route_reports | | 4 | `admin` | 6 | Daily_statistics, exports, system_configuration, transport_factors, support_tickets | | 5 | `integrations` | 3 | Api_cache, weather_data, air_quality_data | | 6 | `notifications` | 3 | Notifications, notification_templates, device_tokens |

5.2.3 Procedimiento de ejecución de migraciones

Paso 1: Verificar estado actual

```
 # Conectar a la BD de producción
psql -h ${DB_HOST} -p ${DB_PORT} -U ${DB_USER} -d ${DB_NAME}

-- Ver changesets aplicados
SELECT id, author, filename, dateexecuted
FROM databasechangelog
ORDER BY orderexecuted DESC
LIMIT 10;

-- Contar total de changesets aplicados
SELECT COUNT(*) as total_changesets FROM databasechangelog;

-- Verificar esquemas existentes
```

```
SELECT schema_name FROM information_schema.schemata
WHERE schema_name NOT IN ('pg_catalog','information_schema','public');
```

Paso 2: Validar changelog (dry run)

```
 # Validar que el changelog es consistente (sin aplicar)
liquibase \
  --changeLogFile=changelog/db.changelog-master.yaml \
  --url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME} \
  --username=${DB_USER} \
  --password=${DB_PASSWORD} \
  validate

# Ver qué cambios se aplicarían (sin ejecutar)
liquibase \
  --changeLogFile=changelog/db.changelog-master.yaml \
  --url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME} \
  --username=${DB_USER} \
  --password=${DB_PASSWORD} \
  status
```

Paso 3: Ejecutar migración

```
 # Aplicar migraciones pendientes
liquibase \
  --changeLogFile=changelog/db.changelog-master.yaml \
  --url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME} \
  --username=${DB_USER} \
  --password=${DB_PASSWORD} \
  update
```

Paso 4: Verificar migración

```
 # Verificar que todos los changesets se aplicaron
SELECT COUNT(*) FROM databasechangelog;  -- Debe ser 77

-- Verificar esquemas
SELECT schema_name FROM information_schema.schemata
WHERE schema_name IN ('security','mobility','community','admin','integrations','notificatio

-- Verificar tablas en cada esquema
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema IN ('security','mobility','community','admin','integrations','notificati
ORDER BY table_schema, table_name;

-- Verificar ENUMs
SELECT t.typname, e.enumlabel
FROM pg_type t
JOIN pg_enum e ON t.oid = e.enumtypid
JOIN pg_namespace n ON t.typnamespace = n.oid
WHERE n.nspname IN ('security','mobility','community','admin','integrations','notifications
ORDER BY t.typname, e.enumsortorder;
```

```
-- Verificar datos seed (roles, permisos)
SELECT COUNT(*) FROM security.roles;          -- Debe ser ≥ 2
SELECT COUNT(*) FROM security.permissions;    -- Debe ser 22
SELECT COUNT(*) FROM admin_transport_factors; -- Debe ser ≥ 4
```

5.2.4 Plan de rollback de migraciones

Si la migración falla, ejecutar el rollback del changeset específico:

```
# Rollback del último changeset
liquibase \
  --changeLogFile=changelog/db.changelog-master.yaml \
  --url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME} \
  --username=${DB_USER} \
  --password=${DB_PASSWORD} \
  rollbackCount 1

# Rollback hasta un changeset específico
liquibase \
  --changeLogFile=changelog/db.changelog-master.yaml \
  --url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME} \
  --username=${DB_USER} \
  --password=${DB_PASSWORD} \
  rollbackToDate <date>
```

En caso de fallo crítico (restauración completa):

```
# 1. Restaurar desde backup pre-despliegue
pg_restore \
  -h ${DB_HOST} -p ${DB_PORT} -U ${DB_USER} -d ${DB_NAME} \
  --clean --if-exists \
  ${BACKUP_FILE}

# 2. Verificar restauración
psql -d ${DB_NAME} -c "SELECT COUNT(*) FROM databasechangelog;"

# 3. Notificar al equipo
echo "ROLLBACK COMPLETADO — Restaurado desde backup $(date)"
```

---

6. Fase 4: Despliegue y configuración del software

6.1 Despliegue del Backend API (Docker → Render)

6.1.1 Construcción de la imagen Docker

```
# Desde el directorio del backend
cd ~/Escritorio/EcoRuteandoBackend

# Build de la imagen
docker build -t ecoruteando-backend:latest .
```

```
# Verificar imagen creada
docker images | grep ecoruteando-backend
```

Dockerfile multi-stage (resumen):

| Etapa | Imagen base | Propósito | |-------|-----------|-----------| | **Build** | `mcr.microsoft.com/dotnet/sdk:9.0` | Compilar la solución completa | | **Runtime** | `mcr.microsoft.com/dotnet/aspnet:9.0` | Ejecutar la API |

6.1.2 Despliegue en Render

| Paso | Acción | Verificación | |------|--------|-------------| | 1 | Push a rama `main` en GitHub | `git log origin/main -1` | | 2 | Render detecta el push y inicia build | Dashboard → Events | | 3 | Build de Docker image | Logs de build sin errores | | 4 | Deploy del contenedor | Estado: `Live` | | 5 | Health check pasa | `/api/test/public` responde 200 | | 6 | Logs de inicio | `"Now listening on: http://[::]:5124"` |

6.1.3 Verificación post-despliegue backend

```
 # Health check
curl -s http://ecoruteando-backend.onrender.com/api/test/public
# Esperado: {"status":"healthy","timestamp":"..."}

# Login con admin seed
curl -s -X POST http://ecoruteando-backend.onrender.com/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@ecoruteando.com","password":"<ADMIN_PASSWORD>"}'
# Esperado: {"token":"eyJ...","refreshToken":"..."}

# Swagger accesible
curl -sI http://ecoruteando-backend.onrender.com/swagger/index.html
# Esperado: HTTP/1.1 200 OK
```

6.2 Despliegue del Frontend Web (Vercel/nginx)

6.2.1 Build de producción

```
 cd ~/Escritorio/EcoRuteando/front-end/ecoRuteando-react

# Instalar dependencias
npm ci

# Build de producción
npm run build

# Verificar build
ls -la dist/
# Debe contener: index.html, assets/
```

6.2.2 Despliegue en Vercel

| Paso | Acción | Verificación | |------|--------|-------------| | 1 | Push a rama `main` | `git log origin/main -1` | | 2 | Vercel detecta push e inicia build | Dashboard → Deployments | | 3 | Build de producción | Logs sin errores | | 4 | Deploy completado | Estado: `Ready` | | 5 | Dominio activo | HTTPS funciona |

6.2.3 Verificación post-despliegue frontend

```
 # Página principal
curl -sI https://<dominio>.vercel.app
# Esperado: HTTP/1.1 200 OK

# Proxy a backend
curl -s https://<dominio>.vercel.app/api/test/public
# Esperado: Respuesta del backend (proxy funciona)

# SPA catch-all
curl -sI https://<dominio>.vercel.app/rutas
# Esperado: HTTP/1.1 200 OK (sirve index.html)
```

6.3 Despliegue de la App Móvil (EAS Build)

6.3.1 Build de producción

```
 cd ~/Escritorio/Movil-EcoRuteando

# Autenticar
eas login

# Build para Android (APK)
eas build --profile production --platform android

# Build para iOS (IPA)
eas build --profile production --platform ios

# Build para ambos
eas build --profile production --platform all
```

6.3.2 Distribución de la app

| Plataforma | Método de distribución | Detalle | |----------|----------------------|---------| | **Android (interno)** | APK directo | Link de descarga de EAS | | **Android (producción)** | Google Play Store | Requiere configuración `submit` | | **iOS (interno)** | TestFlight | Requiere cuenta Apple Developer | | **iOS (producción)** | App Store | Requiere configuración `submit` |

6.3.3 Verificación post-build móvil

```
 # Verificar que el APK se generó
eas build:list --limit 1

# Instalar en dispositivo de prueba
# (Android) Descargar APK desde link de EAS
adb install path/to/app.apk
```

```
# Verificar conexión al backend
# Abrir app → Login → Planificar ruta
```

6.4 Configuración post-despliegue

6.4.1 Configuración del administrador

| Paso | Acción | SQL/API | |------|--------|---------| | 1 | Verificar que el admin seed existe | `SELECT * FROM security.users WHERE email = 'admin@ecoruteando.com';` | | 2 | Verificar rol Admin asignado | `SELECT * FROM security.user_roles WHERE user_id = <admin_id>;` | | 3 | Verificar permisos del rol Admin | `SELECT * FROM security.role_permissions WHERE role_id = <admin_role_id>;` | | 4 | Login exitoso como admin | `POST /api/auth/login` |

6.4.2 Configuración de políticas de seguridad

| Parámetro | Valor por defecto | Verificación | |-----------|-------------------|-------------| | Longitud mínima de contraseña | 8 caracteres | `SELECT * FROM security.security_policies;` | | Intentos máximos de login | 5 | Igual que arriba | | Duración de bloqueo | 30 minutos | Igual que arriba | | Expiración de refresh token | 7 días | Configuración en backend | | Expiración de access token | 60 minutos | `Jwt__ExpirationMinutes` |

---

7. Fase 5: Pruebas de aceptación y verificación

7.1 Pruebas de humo (Smoke Tests)

Se ejecutan inmediatamente después del despliegue para verificar que los componentes básicos funcionan.

7.1.1 Pruebas de salud del sistema

| # | Prueba | Comando / Acción | Criterio de aceptación | Estado | |---|--------|------------------|------------------------|--------| | S-01 | Backend responde | `GET /api/test/public` | HTTP 200 con JSON | ⬜ | | S-02 | Frontend carga | Abrir URL en navegador | Página de login visible | ⬜ | | S-03 | PostgreSQL acepta conexiones | `pg_isready` | `accepting connections` | ⬜ | | S-04 | Liquibase completado | `SELECT COUNT(*) FROM databasechangelog` | 77 changesets | ⬜ | | S-05 | Swagger accesible | `GET /swagger/index.html` | HTTP 200 | ⬜ |

7.1.2 Pruebas de autenticación

| # | Prueba | Acción | Criterio de aceptación | Estado | |---|--------|--------|------------------------|--------| | S-06 | Login admin válido | `POST /api/auth/login` con credenciales admin | HTTP 200 con token JWT | ⬜ | | S-07 | Login inválido rechazado | `POST /api/auth/login` con contraseña incorrecta | HTTP 401 | ⬜ | | S-08 | Endpoint protegido con token | `GET /api/auth/me` con token válido | HTTP 200 con datos usuario | ⬜ | | S-09 | Endpoint protegido sin token | `GET /api/auth/me` sin token | HTTP 401 | ⬜ | | S-10 | Refresh token funciona | `POST /api/auth/refresh` | HTTP 200 con nuevo token | ⬜ |

7.2 Pruebas de integración geoespacial

| # | Prueba | Acción | Criterio de aceptación | Estado | |---|--------|--------|----------------------|--------| | G-01 | Planificación de ruta | `POST /api/routes` con origen/destino | HTTP 201 con geometría | ⬜ | | G-02 | Google Maps Directions | `GET /api/maps/directions` | HTTP 200 con polyline | ⬜ | | G-03 | Geocodificación | `GET /api/maps/geocode` | HTTP 200 con coordenadas | ⬜ | | G-04 | Estimación CO_2 | `GET /api/sustainability/estimate` | HTTP 200 con valor CO_2 | ⬜ | | G-05 | Cálculo calorías | Igual que G-04 | HTTP 200 con calorías | ⬜ | | G-06 | POIs cercanos | `GET /api/pois` | HTTP 200 con lista | ⬜ | | G-07 | Reporte obstáculos | `POST /api/obstacle-reports` con GPS | HTTP 201 con ubicación | ⬜ | | G-08 | Clima por ruta | `GET /api/weather/route` | HTTP 200 con datos clima | ⬜ |

7.3 Pruebas de integración con base de datos

| # | Prueba | SQL / Acción | Criterio de aceptación | Estado | |---|--------|-------------|----------------------|--------| | D-01 | 6 esquemas existen | `SELECT schema_name FROM information_schema.schemata` | 6 esquemas | ⬜ | | D-02 | 36 tablas creadas | `SELECT COUNT(*) FROM information_schema.tables` | ≥ 36 (excluyendo catálogos) | ⬜ | | D-03 | 17 ENUMs creados | `SELECT COUNT(DISTINCT typname) FROM pg_type...` | ≥ 17 | ⬜ | | D-04 | Roles sembrados | `SELECT * FROM security.roles` | ≥ 2 (User, Admin) | ⬜ | | D-05 | Permisos sembrados | `SELECT COUNT(*) FROM security.permissions` | 22 | ⬜ | | D-06 | Factores transporte | `SELECT COUNT(*) FROM admin.transport_factors` | ≥ 4 | ⬜ | | D-07 | PostGIS habilitado | `SELECT PostGIS_Version()` | Versión 3.x | ⬜ | | D-08 | Columnas geoespaciales | `SELECT * FROM geometry_columns` | Múltiples columnas | ⬜ |

7.4 Pruebas de aceptación de usuario (UAT)

7.4.1 Flujo de registro y verificación

| # | Paso del usuario | Acción | Resultado esperado | Estado | |---|-----------------|--------|-------------------|--------| | U-01 | Registro | Completar formulario de registro | Cuenta creada, email de verificación enviado | ⬜ | | U-02 | Verificación email | Click en enlace de verificación | Cuenta verificada, mensaje de éxito | ⬜ | | U-03 | Login | Ingresar credenciales | Acceso al dashboard | ⬜ |

7.4.2 Flujo de planificación de rutas

| # | Paso del usuario | Acción | Resultado esperado | Estado | |---|-----------------|--------|-------------------|--------| | U-04 | Planificar ruta | Ingresar origen y destino | Ruta mostrada en mapa | ⬜ | | U-05 | Seleccionar transporte | Elegir tipo (caminando/carro/bici) | Ruta recalculada | ⬜ | | U-06 | Ver estimaciones | Revisar panel de CO_2 y calorías | Valores mostrados | ⬜ | | U-07 | Guardar ruta | Click en "Guardar" | Ruta en "Mis rutas" | ⬜ | | U-08 | Marcar favorito | Click en estrella | Favorito agregado | ⬜ |

7.4.3 Flujo de reporte de obstáculos

| # | Paso del usuario | Acción | Resultado esperado | Estado | |---|-----------------|--------|-------------------|--------| | U-09 | Ver obstáculos | Abrir mapa de reportes | Marcadores visibles | ⬜ | | U-10 | Crear reporte | Tomar foto + ubicación GPS | Reporte creado | ⬜ | | U-11 | Votar reporte | Votar "útil" en reporte existente | Voto registrado | ⬜ |

7.4.4 Flujo de administración

| # | Paso del usuario | Acción | Resultado esperado | Estado | |---|---|---|---|---| | U-12 | Panel admin | Login como admin → Panel | Dashboard con estadísticas | ⬜ | | U-13 | Gestionar usuarios | Listar, editar usuario | Cambios aplicados | ⬜ | | U-14 | Moderar reportes | Validar/rechazar obstáculo | Estado actualizado | ⬜ | | U-15 | Exportar datos | Solicitar exporte CSV | Archivo descargable | ⬜ |

7.4.5 Flujo de aplicación móvil

| # | Paso del usuario | Acción | Resultado esperado | Estado | |---|---|---|---|---| | U-16 | Login móvil | Abrir app → Login | Acceso correcto | ⬜ | | U-17 | Planificar ruta móvil | Usar planificador | Ruta en mapa móvil | ⬜ | | U-18 | Perfil con foto | Subir foto de perfil | Foto visible y persistente | ⬜ | | U-19 | Modo invitado | Entrar como invitado | Funcionalidad limitada | ⬜ |

7.5 Registro de resultados de pruebas

| ID | Prueba | Ejecutada por | Fecha | Resultado | Observaciones | |---|---|---|---|---|---| | S-01 | Backend responde | | | ⬜ Pass / ⬜ Fail | | | S-02 | Frontend carga | | | ⬜ Pass / ⬜ Fail | | | ... | ... | | | ... | |

Criterio global de aceptación:

- ✅ **Aceptado:** 100% de pruebas de humo (S-01 a S-10) con resultado PASS
- ✅ **Aceptado:** ≥ 95% de pruebas de integración (G-01 a G-08, D-01 a D-08) con resultado PASS
- ✅ **Aceptado:** 100% de flujos UAT críticos (U-01 a U-08) con resultado PASS
- ⚠️ **Condicional:** ≤ 5% de pruebas no críticas con FAIL, con plan de remediación
- ❌ **Rechazado:** Cualquier prueba de humo con FAIL

---

8. Fase 6: Capacitación y entrega formal

8.1 Plan de capacitación

8.1.1 Audiencias de capacitación

| # | Audiencia | Nº de sesiones | Duración | Contenido | |---|---|---|---|---| | C-01 | Usuarios finales (público general) | 2 | 1.5 horas c/u | Registro, planificación de rutas, favoritos, reportes | | C-02 | Administradores del sistema | 3 | 2 horas c/u | Panel admin, gestión de usuarios, moderación, exportes | | C-03 | Personal de soporte técnico | 2 | 2 horas c/u | Troubleshooting, reset de contraseñas, logs | | C-04 | Equipo de desarrollo (interno) | 4 | 3 horas c/u | Arquitectura, código, despliegue, mantenimiento |

8.1.2 Contenido detallado por sesión

C-01: Usuarios finales

| Sesión | Tema | Contenido | |--------|------|-----------| | 1 | Introducción y registro | Creación de cuenta, verificación de email, login | | 1 | Planificación de rutas | Uso del planificador, selección de transporte, estimaciones | | 2 | Funciones sociales | Favoritos, calificaciones, compartir rutas | | 2 | Reporte de obstáculos | Creación de reportes con foto y GPS, votación |

C-02: Administradores

| Sesión | Tema | Contenido | |--------|------|-----------| | 1 | Panel de administración | Dashboard, estadísticas, métricas clave | | 1 | Gestión de usuarios | Listar, editar, desactivar, promover roles | | 2 | Moderación de reportes | Validar, rechazar, resolver obstáculos | | 2 | Exportes y reportes | Generación de CSV/Excel, filtros | | 3 | Configuración del sistema | Factores de transporte, políticas de seguridad |

8.1.3 Recursos de capacitación

| Recurso | Formato | Disponibilidad | |---------|---------|---------------| | Manual de usuario | PDF / Web | Portal de ayuda | | Video-tutoriales | MP4 / YouTube | Canal privado | | Guía rápida (quick reference) | PDF de 1 página | Descarga desde app | | FAQ (preguntas frecuentes) | Web | Portal de ayuda | | Sesiones en vivo | Videoconferencia | Calendarización previa |

8.2 Acta de entrega formal

8.2.1 Estructura del acta de entrega

ACTA DE ENTREGA E IMPLANTACIÓN DEL SOFTWARE

Proyecto: EcoRuteando — Plataforma de Movilidad Sostenible

| Campo | Valor | |-------|-------| | **Fecha de entrega** | **//**_____ | | **Versión del software** | 1.0 | | **Entorno de implantación** | Producción |

1. Componentes entregados

| # | Componente | Versión | Estado | |---|-----------|---------|--------| | 1 | Backend API (.NET 9) | 1.0 | ⬜ Entregado | | 2 | Base de Datos (PostgreSQL + PostGIS) | 16 + 3.5 | ⬜ Entregada | | 3 | Frontend Web (React + Vite) | 1.0 | ⬜ Entregado | | 4 | Aplicación Móvil (Expo) | 1.0 | ⬜ Entregada | | 5 | Migraciones de Base de Datos (Liquibase) | 77 changesets | ⬜ Aplicadas |

2. Criterios de aceptación verificados

| # | Criterio | Resultado | |---|---------|-----------| | 1 | Sistema disponible en producción | ⬜ Cumple / ⬜ No cumple | | 2 | Autenticación funcional (email + Google) | ⬜ Cumple / ⬜ No cumple | | 3 | Planificación de rutas operativa | ⬜ Cumple / ⬜ No cumple | | 4 | Panel de administración funcional | ⬜ Cumple / ⬜ No cumple | | 5 | Base de datos con esquema completo | ⬜ Cumple / ⬜ No cumple | | 6 | Pruebas de humo aprobadas (100%) | ⬜ Cumple / ⬜ No cumple | | 7 | Pruebas de integración aprobadas (≥95%) | ⬜ Cumple / ⬜ No cumple | | 8 | UAT aprobado por el cliente | ⬜ Cumple / ⬜ No cumple | | 9 | Capacitación completada | ⬜ Cumple / ⬜ No cumple | | 10 | Documentación entregada | ⬜ Cumple / ⬜ No cumple |

3. Documentación entregada

| # | Documento | Versión | Estado | |---|-----------|---------|--------| | 1 | Manual Técnico | 1.0 | ⬜ Entregado | | 2 | Manual de Instalación | 1.0 | ⬜ Entregado | | 3 | Procedimiento de Implantación | 1.0 | ⬜ Entregado | | 4 | Documento de Acciones Correctivas/Preventivas | 1.0 | ⬜ Entregado | | 5 | Manual de Usuario | 1.0 | ⬜ Entregado |

4. Credenciales y accesos entregados

| # | Recurso | Ubicación de credenciales | Estado | |---|---------|---------------------------|--------| | 1 | Acceso a Render (backend + BD) | Administrado por el cliente | ⬜ Entregado | | 2 | Acceso a Vercel (frontend) | Administrado por el cliente | ⬜ Entregado | | 3 | Acceso a Expo/EAS (móvil) | Administrado por el cliente | ⬜ Entregado | | 4 | Google Cloud Console | Administrado por el cliente | ⬜ Entregado | | 5 | Cuenta administrador del sistema | Canal seguro | ⬜ Entregado |

5. Observaciones y pendientes

| # | Observación | Responsable | Fecha compromiso | |---|-------------|-------------|------------------| | 1 | | | | | 2 | | | | | 3 | | | |

6. Firma de conformidad

| Rol | Nombre | Firma | Fecha | |-----|--------|-------|-------| | **Entregante (DevOps / Tech Lead)** | _____ | _____ | //_____ | | **Receptor (Cliente / Gestor)** | _____ | _____ | //_____ | | **Testigo (QA Engineer)** | _____ | _____ | //_____ |

---

9. Fase 7: Mantenimiento, soporte post-implantación y monitoreo continuo

9.1 Niveles de soporte

| Nivel | Descripción | Tiempo de respuesta | Tiempo de resolución | Horario | |-------|-------------|---------------------|----------------------|---------| | **Nivel 1** | Consultas de usuarios, uso del sistema | 24 horas | 48 horas | Laborales 8:00-18:00 | | **Nivel 2** | Incidencias técnicas menores | 8 horas | 24 horas | Laborales 8:00-18:00 | | **Nivel 3** | Fallos críticos (sistema caído) | 1 hora | 4 horas | 24/7 | | **Nivel 4** | Emergencias de seguridad | 30 minutos | 2 horas | 24/7 |

9.2 Monitoreo continuo

9.2.1 Herramientas de monitoreo

| Componente | Herramienta | Métrica | Frecuencia | |------------|-------------|---------|------------| | **Backend (uptime)** | Uptime Robot / BetterStack | Disponibilidad HTTP | Cada 1 min | | **Backend (rendimiento)** | Render Metrics | CPU, RAM, response time | Continuo | | **Base de datos** | Render Dashboard | Conexiones, storage, IOPS | Continuo | | **Frontend** | Vercel Analytics | Visitas, errores, performance | Continuo | | **API externas** | Google Cloud Console | Cuota, errores, latencia | Diario | |

Logs | Serilog + Seq/ELK | Errores, warnings, audit | Continuo | | **Errores** | Sentry (opcional) | Excepciones no capturadas | Tiempo real |

9.2.2 Dashboard de métricas operativas

| Métrica | Objetivo | Alerta temprana | Alerta crítica | |---------|----------|----------------|---------------| | Uptime del backend | ≥ 99.5% | < 99% | < 95% | | Tiempo de respuesta API (promedio) | < 500ms | > 1s | > 3s | | Tasa de errores 5xx | < 0.1% | > 1% | > 5% | | Conexiones a BD | < 80% del máximo | > 80% | > 95% | | Uso de almacenamiento BD | < 70% | > 80% | > 90% | | Cuota Google Maps | < 60% | > 80% | > 95% |

9.2.3 Alertas configuradas

| Alerta | Condición | Canal | Responsable | |--------|----------|-------|-------------| | Backend caído | Health check falla 3 veces consecutivas | Email + Slack | DevOps | | Errores 5xx | > 1% de requests en 5 min | Email | DevOps + Tech Lead | | Cuota Google Maps | > 80% consumida | Email | DevOps | | Backup diario fallido | Script de backup retorna error | Email | DevOps | | Login fallidos | > 50 intentos fallidos en 1 hora | Email | Security | | Espacio BD | > 80% consumido | Email | DevOps |

9.3 Mantenimiento programado

| Actividad | Frecuencia | Responsable | Duración | |-----------|-----------|-------------|----------| | Revisión de logs de errores | Diaria | DevOps | 15 min | | Verificación de backups | Diaria | DevOps | 10 min | | Revisión de métricas de rendimiento | Semanal | DevOps + Tech Lead | 30 min | | Actualización de dependencias (patch) | Semanal | Tech Lead | 1 hora | | Revisión de seguridad (vulnerabilidades) | Mensual | Security | 2 horas | | Rotación de API keys | Cada 90 días | DevOps | 30 min | | Restore de prueba desde backup | Trimestral | DevOps | 1 hora | | Auditoría de accesos | Trimestral | Security | 2 horas | | Actualización de PostgreSQL | Semestral | DevOps | Ventana mantenimiento | | Revisión completa del sistema | Anual | Equipo completo | 1 día |

9.4 Gestión de incidencias

9.4.1 Proceso de gestión

```
 1. DETECCIÓN → Alerta automática o reporte de usuario
2. CLASIFICACIÓN → Severidad (Crítica/Alta/Media/Baja)
3. ASIGNACIÓN → Responsable según severidad
4. DIAGNÓSTICO → Identificación de causa raíz
5. RESOLUCIÓN → Implementación de fix
6. VERIFICACIÓN → Confirmación de resolución
7. CIERRE → Documentación y lecciones aprendidas
```

9.4.2 Matriz de severidad

| Severidad | Descripción | Ejemplos | Tiempo respuesta | Tiempo resolución | |-----------|------------|----------|-----------------|------------------| | **S1 - Crítica** | Sistema caído, datos comprometidos | Backend no responde, breach de seguridad | 30 min | 2 horas | | **S2 - Alta** | Funcionalidad principal degradada | Login falla, rutas no se calculan | 2 horas | 8 horas | | **S3 - Media** | Funcionalidad secundaria

afectada | Notificaciones no llegan, exporte falla | 8 horas | 24 horas | | **B4 - Baja** | Inconveniente menor | Error visual, texto incorrecto | 24 horas | 72 horas |

9.5 Plan de continuidad operativa (BCP)

| Escenario | Impacto | Plan de recuperación | RTO | RPO | |----------|---------|---------------------|-----|----- | | **Caída del backend** | Alto | Auto-restart (`restart: unless-stopped`); si persiste, redeploy manual | 15 min | 0 | | **Caída de la BD** | Crítico | Failover a réplica; restaurar desde backup si es necesario | 30 min | 15 min | | **Caída de Google Maps** | Alto | Circuit breaker + caché; cálculo local con Haversine | Degradado | N/A | | **Compromiso de seguridad** | Crítico | Rotar credenciales, auditar logs, bloquear accesos comprometidos | 1 hora | 0 | | **Pérdida de datos** | Crítico | Restaurar desde último backup verificado | 30 min | 24 horas |

9.6 Escalamiento

```
 Nivel 1 (DevOps)
   │  → No resuelto en 2 horas
   ▼
Nivel 2 (Tech Lead)
   │  → No resuelto en 4 horas
   ▼
Nivel 3 (Arquitecto + Cliente)
   │  → No resuelto en 8 horas
   ▼
Nivel 4 (Equipo completo + Stakeholders)
```

Anexo A: Checklist completa de implantación

Pre-implantación

- [ ] Ventana de mantenimiento aprobada por el cliente
- [ ] Notificación enviada a usuarios (24h antes)
- [ ] Backup completo realizado y verificado
- [ ] Variables de entorno configuradas en Render
- [ ] Variables de entorno configuradas en Vercel
- [ ] Secretos rotados si es necesario
- [ ] API keys de Google verificadas activas
- [ ] SMTP verificado funcional

Durante la implantación

- [ ] Backup pre-despliegue ejecutado
- [ ] Migraciones Liquibase aplicadas (77/77)
- [ ] Backend desplegado y health check OK
- [ ] Frontend desplegado y accesible
- [ ] Proxy `/api/*` funcionando en Vercel

Post-implantación

- [ ] Pruebas de humo (S-01 a S-10) — 100% PASS
- [ ] Pruebas de integración geoespacial (G-01 a G-08) — ≥95% PASS
- [ ] Pruebas de BD (D-01 a D-08) — 100% PASS
- [ ] UAT ejecutado y aprobado por el cliente
- [ ] App móvil construida y distribuida
- [ ] Capacitación realizada
- [ ] Acta de entrega firmada
- [ ] Monitoreo activo y alertas configuradas
- [ ] Soporte post-implantación activo

Procedimiento de Implantación del Software — EcoRuteando — v1.0 — 13 de octubre de 2026